

¿Qué es Jest y React Testing Library? ¿Cuáles son sus características y para qué sirve? Desarrollar un ejemplo sencillo en código REACT que permita visualizar lo descrito, explicando el ejemplo.

Son dos librerías estándar para crear pruebas unitarias y de integración dentro del ecosistema de React.

Jest nos provee el entorno de ejecución de pruebas, la estructura para organizar suites, aserciones, y utilidades para simular datos o funciones. Por otro lado, React Testing Library nos proporciona las herramientas para renderizar componentes de React en un DOM virtual y poder interactuar con ellos tal cual lo hace un usuario final.

```
import { useState } from 'react';

export default function ContadorTest() {
  const [contador, setContador] = useState(0);

  return (
    <div>
      <h2>Valor: {contador}</h2>
      <button onClick={() => setContador(contador + 1)}>Sumar</button>
      <button onClick={() => setContador(0)}>Reiniciar</button>
    </div>
  );
}
```

Supongamos que nosotros tenemos este componente el cual es un contador que se incrementa de a uno o se reinicia en caso de que el usuario haga click en los botones sumar o reiniciar respectivamente. En el archivo de prueba podríamos tener algo así como:

```

import { render, screen, fireEvent } from '@testing-library/react';
import ContadorTest from './ContadorTest';

describe('Pruebas sobre ContadorTest', () => {
  test('debe mostrar el valor inicial en 0', () => {
    render(<ContadorTest />);

    // Busca un encabezado que contenga el texto inicial esperado
    const heading = screen.getByRole('heading', { name: /valor: 0/i });
    expect(heading).toBeInTheDocument();
  });

  test('debe incrementar el contador al hacer clic en Sumar', () => {
    render(<ContadorTest />);

    // Obtener el botón por su rol accesible y nombre visible
    const botonSumar = screen.getByRole('button', { name: /sumar/i });

    // Simular el clic del usuario
    fireEvent.click(botonSumar);

    // Verificar que el DOM cambió como se espera
    const headingActualizado = screen.getByRole('heading', { name: /valor: 1/i });
    expect(headingActualizado).toBeInTheDocument();
  });

  test('debe reiniciar el contador a 0', () => {
    render(<ContadorTest />);

    const botonSumar = screen.getByRole('button', { name: /sumar/i });
    const botonReiniciar = screen.getByRole('button', { name: /reiniciar/i });

    // Dos clics para subir a 2 y luego reiniciar
    fireEvent.click(botonSumar);
    fireEvent.click(botonSumar);
    fireEvent.click(botonReiniciar);

    expect(screen.getByRole('heading', { name: /valor: 0/i })).toBeInTheDocument();
  });
});

```

En donde:

- `render(<ContadorTest/>)`: RTL monta el componente en el entorno simulado de Node/JSDOM provisto por Jest.
- `screen.getByRole(...)`: Busca elementos de forma similar a como un lector de pantalla o un usuario real los identifica (en este caso, botones con sus etiquetas visibles). Si el elemento no existe, el test falla automáticamente aquí.
- `fireEvent.click(...)`: Envía un evento sintético de clic al botón, provocando la ejecución de la función asociada en `onClick` y el consecuente cambio de estado.
- `expect(...).toBeInTheDocument()`: Aserción ejecutada por Jest que comprueba si el nodo del DOM actualizado se encuentra presente en el documento virtual.